



Container Lifecycle Notes

Understand what happens with containers throughout their lifetime.

1. Getting Started (Creating & Running)

*To get a container into the **Container running** state, you can use:*

👉 `docker run <image>`

👉 OR a two-step process: `docker create` + `docker start`

2. Container Running State

While a container is actively running, you can interact with it using:

👉 `docker logs`

👉 `docker inspect`

👉 `docker exec`

3. Actions from the Running State

| *(Assuming no restart policy is in place)*

Pausing:

👉 *Command:* `docker pause <cId>`

👉 *Result:* Container is not truly stopped + Keeps memory contents.

👉 *To reverse:* Use `docker unpause <cId>` to go back to running.

Graceful Stop:

👉 *Command:* `docker stop <cId>`

👉 *Result:* Container stops gracefully + Clears memory contents ➡ Goes to Container stopped.

Force Stop:

👉 Command: `docker kill <cId>`

👉 Result: Container is forced to stop with `SIGKILL` (default) → Goes to Container stopped.

Exiting (Self-termination):

👉 `exit 0` → Container exited without an error → Goes to Container stopped.

👉 `exit <non-zero>` → Container exited with an error → Goes to Container stopped.

4. 🛑 Container Stopped State

When a container is stopped, it is not deleted!

👉 Visibility: The container still exists and can be seen using `docker ps -a`.

👉 Available Commands: You can still use `docker logs` and `docker inspect`.

👉 To restart: Run `docker start <cId>` to push it back to the Container running state.

5. 🗑️ Container Removed State

From the stopped state, you can permanently delete the container.

👉 Command: `docker rm <cId>`

👉 Result: Container and its contents are fully removed and are no longer available.